Georgia State University

CSC 4520/6520
Design & Analysis of Algorithms

CHAPTER 5: DIVIDE-AND-CONQUER

Abdullah Bal, PhD

Objectives

Recap

Divide-and-Conquer

Master Theorem

Mergesort

Quicksort

Hoare's Partitioning

Multiplication of large integers

Strassen's matrix multiplication

Closest-Pair Problem

Multiplication of Large Integers

- Some applications, notably modern cryptography, require manipulation of integers that are over 100 decimal digits long.
- Since such integers are too long to fit in a single word of a modern computer, they require special treatment.
- if we use the conventional pen-and-pencil algorithm for multiplying two n-digit integers, each of the n digits of the first number is multiplied by each of the n digits of the second number for the total of n^2 digit multiplications.
- Though it might appear that it would be impossible to design an algorithm with fewer than n^2 digit multiplications, this turns out not to be the case.

Multiplication of Large Integers

- Consider the problem of multiplying two (large) *n*-digit integers represented by arrays of their digits such as:

  A = 1234567890135798642 9 B = 876543212848209128 36

  The grade-school algorithm:

$$
\begin{array}{r}
a_1\ a_2 \ldots\ a_n \\
b_1\ b_2 \ldots\ b_n \\
\hline
(d_{10})\ d_{11} d_{12} \ldots\ d_{1n} \\
(d_{20})\ d_{21} d_{22} \ldots\ d_{2n} \\
\ldots\ \ldots\ \ldots\ \ldots\ \ldots\ \ldots\ \ldots \\
\hline
(d_{n0})\ d_{n1} d_{n2} \ldots\ d_{nn}
\end{array}
$$

- Efficiency: n^2 one-digit multiplications

First Step: Divide-and-Conquer Algorithm

- A small example: $A * B$ where A = 2135 and B = 4014

$$A = (21 \cdot 10^2 + 35),$$

$$B = (40 \cdot 10^2 + 14)$$

So,

$$A * B = (21 \cdot 10^2 + 35) * (40 \cdot 10^2 + 14)$$

$$= 21 * 40 \cdot 10^4 + (21 * 14 + 35 * 40) \cdot 10^2 + 35 * 14$$

In general, if $A = A_1A_2$ and $B = B_1B_2$ (where A and B are *n*-digit, A_1, A_2, B_1, B_2 are *n/2*-digit numbers),

$$A * B = A_1 * B_1 \cdot 10^n + (A_1 * B_2 + A_2 * B_1) \cdot 10^{n/2} + A_2 * B_2.$$ (4 Multiplications and 3 add)

First Step: Divide-and-Conquer Algorithm

$A * B = A_1 * B_1 \cdot 10^n + (A_1 * B_2 + A_2 * B_1) \cdot 10^{n/2} + A_2 * B_2$ (4 Multiplications and 3 add)

- Recurrence for the number of one-digit multiplications M(n):

$$M(n) = 4M(n/2), \quad M(1) = 1$$

- Solution: $M(n) = n^2$

Second Step: Divide-and-Conquer Algorithm

- The idea is to decrease the number of multiplications from 4 to 3:

$$A * B = A_1 * B_1 \cdot 10^n + (A_1 * B_2 + A_2 * B_1) \cdot 10^{n/2} + A_2 * B_2 \quad \text{(4 Multiplications and 3 add)}$$
$$= c_2 \cdot 10^n + c_1 \cdot 10^{n/2} + c_0$$

$c_2 = A_1 * B_1$

$c_0 = A_2 * B_2$

$c_1 = A_1 * B_2 + A_2 * B_1$

- Rewrite c_1

$(A_1 + A_2) * (B_1 + B_2) = A_1 * B_1 + (A_1 * B_2 + A_2 * B_1) + A_2 * B_2$,

i.e., $(A_1 * B_2 + A_2 * B_1) = (A_1 + A_2) * (B_1 + B_2) - A_1 * B_1 - A_2 * B_2$,

$c_1 = (A_1 + A_2) * (B_1 + B_2) - (c_2 + c_0)$

- Requires only 3 multiplications at the expense of extra add/sub.

Second Step: Divide-and-Conquer Algorithm

- Recurrence for the number of multiplications M(n):

$$M(n) = 3M(n/2), \quad M(1) = 1$$

- Solution: Backward substation $M(n) = 3^{\log_2 n} = n^{\log_2 3} \approx n^{1.585}$

Master Theorem
If $a < b^d$, $T(n) \in \Theta(n^d)$
If $a = b^d$, $T(n) \in \Theta(n^d \log n)$
If $a > b^d$, $T(n) \in \Theta(n^{\log_b a})$

($a^{\log_b c} = c^{\log_b a}$)

Strassen's Matrix Multiplication

- Strassen observed [1969] that the product of two matrices can be computed as follows:

$$\begin{pmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{pmatrix} = \begin{pmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{pmatrix} * \begin{pmatrix} B_{00} & B_{01} \\ B_{10} & B_{11} \end{pmatrix}$$

$$C_{00}=A_{00}*B_{00} + A_{01}*B_{10}$$
$$C_{01}=A_{00}*B_{01} + A_{01}*B_{11}$$
$$C_{10}=A_{10}*B_{00} + A_{11}*B_{10}$$
$$C_{11}=A_{10}*B_{01} + A_{11}*B_{11}$$

Formulas for Strassen's Algorithm

$M_1 = (A_{00} + A_{11}) * (B_{00} + B_{11})$

$M_2 = (A_{10} + A_{11}) * B_{00}$

$M_3 = A_{00} * (B_{01} - B_{11})$

$M_4 = A_{11} * (B_{10} - B_{00})$

$M_5 = (A_{00} + A_{01}) * B_{11}$

$M_6 = (A_{10} - A_{00}) * (B_{00} + B_{01})$

$M_7 = (A_{01} - A_{11}) * (B_{10} + B_{11})$

Strassen's Matrix Multiplication

- Strassen presented the product of two matrices using Strassen formulas:

$$\begin{pmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{pmatrix} = \begin{pmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{pmatrix} * \begin{pmatrix} B_{00} & B_{01} \\ B_{10} & B_{11} \end{pmatrix}$$

$$C_{00}=A_{00}*B_{00} + A_{01}*B_{10}$$
$$C_{01}=A_{00}*B_{01} + A_{01}*B_{11}$$
$$C_{10}=A_{10}*B_{00} + A_{11}*B_{10}$$
$$C_{11}=A_{10}*B_{01} + A_{11}*B_{11}$$

$$= \begin{pmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 + M_3 - M_2 + M_6 \end{pmatrix}$$

Analysis of Strassen's Algorithm

- If n is not a power of 2, matrices can be padded with zeros.
- Number of multiplications:

$$M(n) = 7M(n/2), \quad M(1) = 1$$

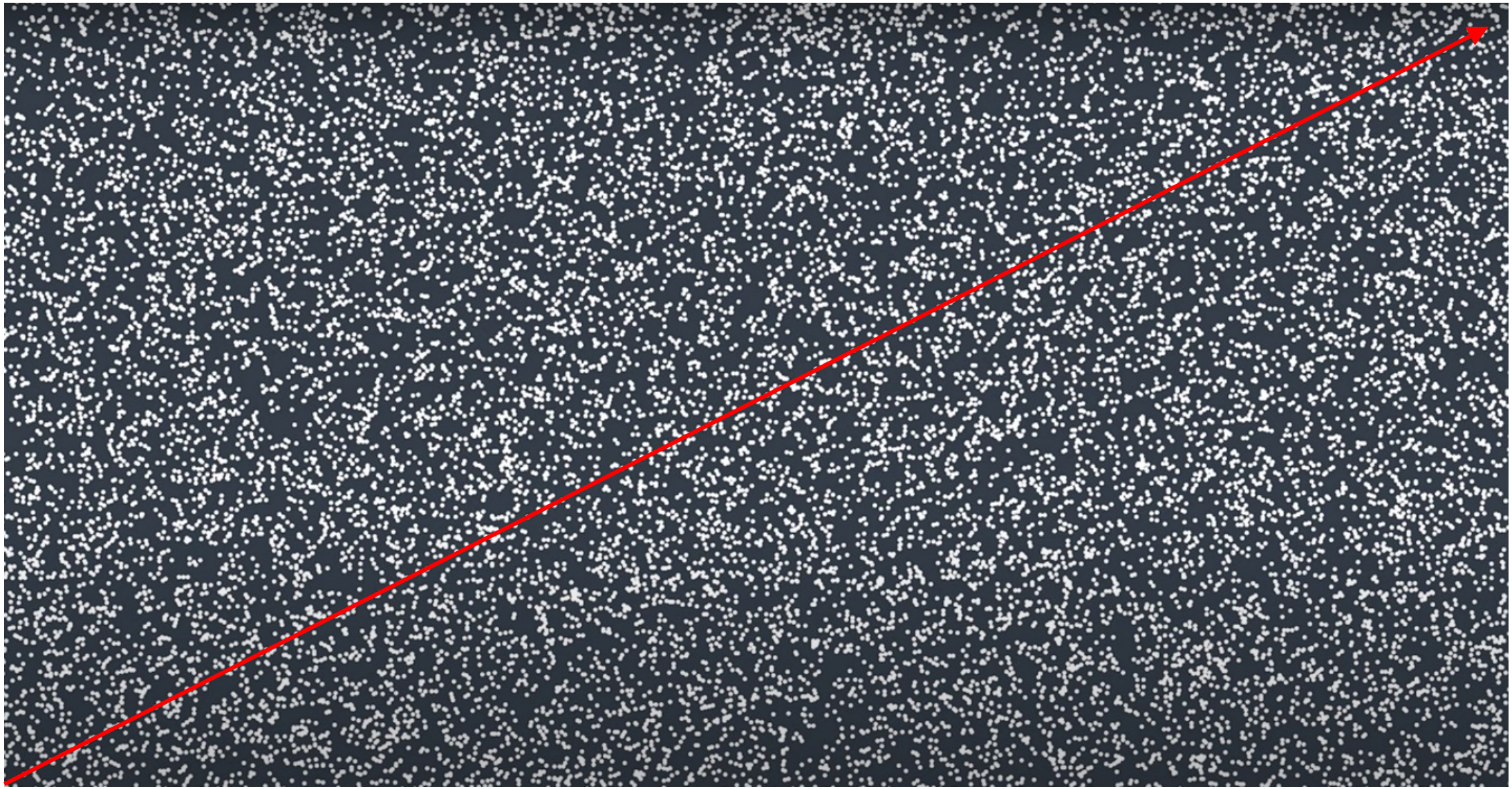
- Solution: $M(n) = 7^{\log_2 n} = n^{\log_2 7} \approx n^{2.807}$ vs. n^3 of brute-force algorithm
- Algorithms with better asymptotic efficiency are known but they are even more complex.

Closest-Pair Problem by Divide-and-Conquer

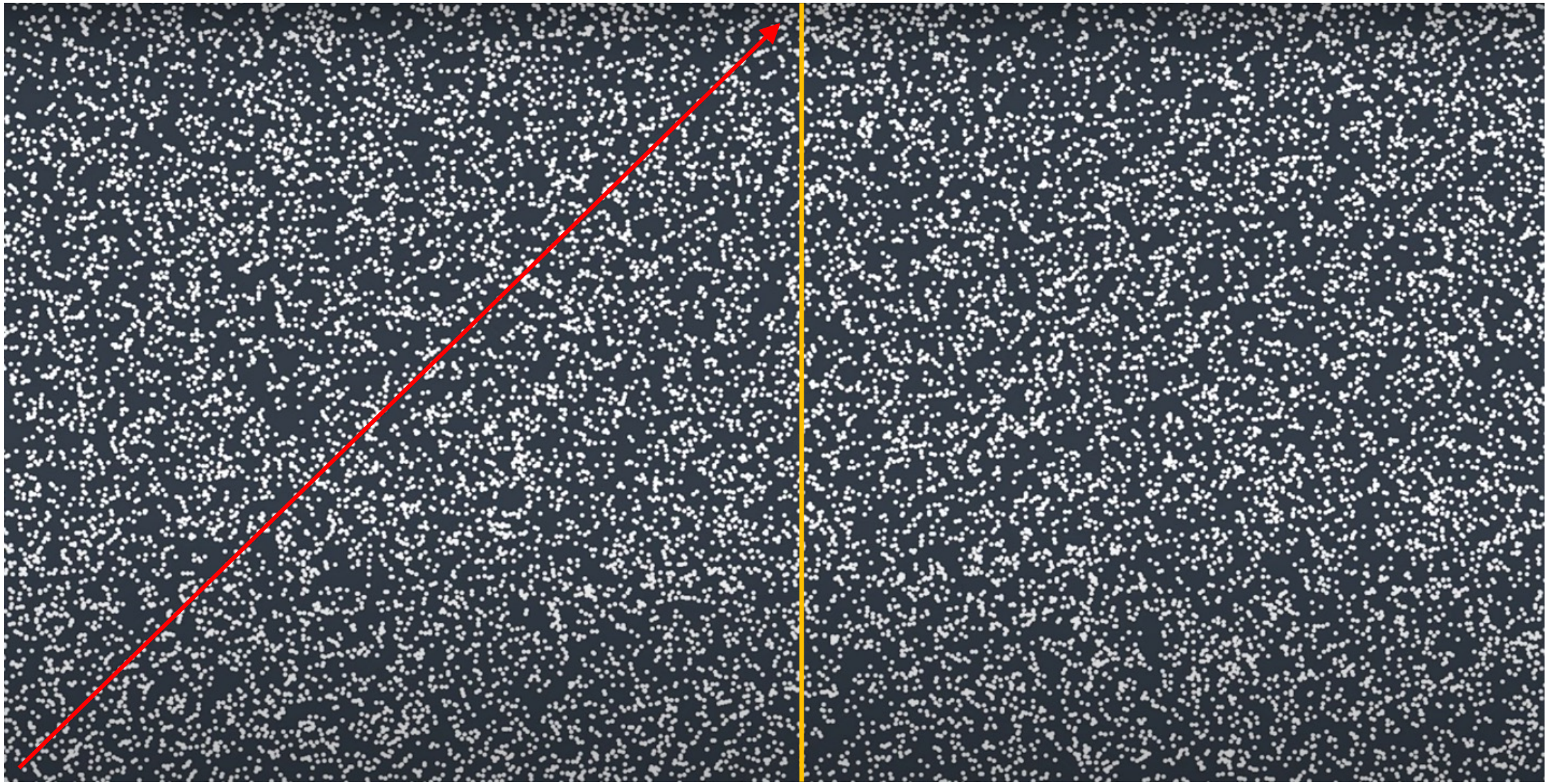
Closest-Pair Problem by Divide-and-Conquer

Closest-Pair Problem by Divide-and-Conquer

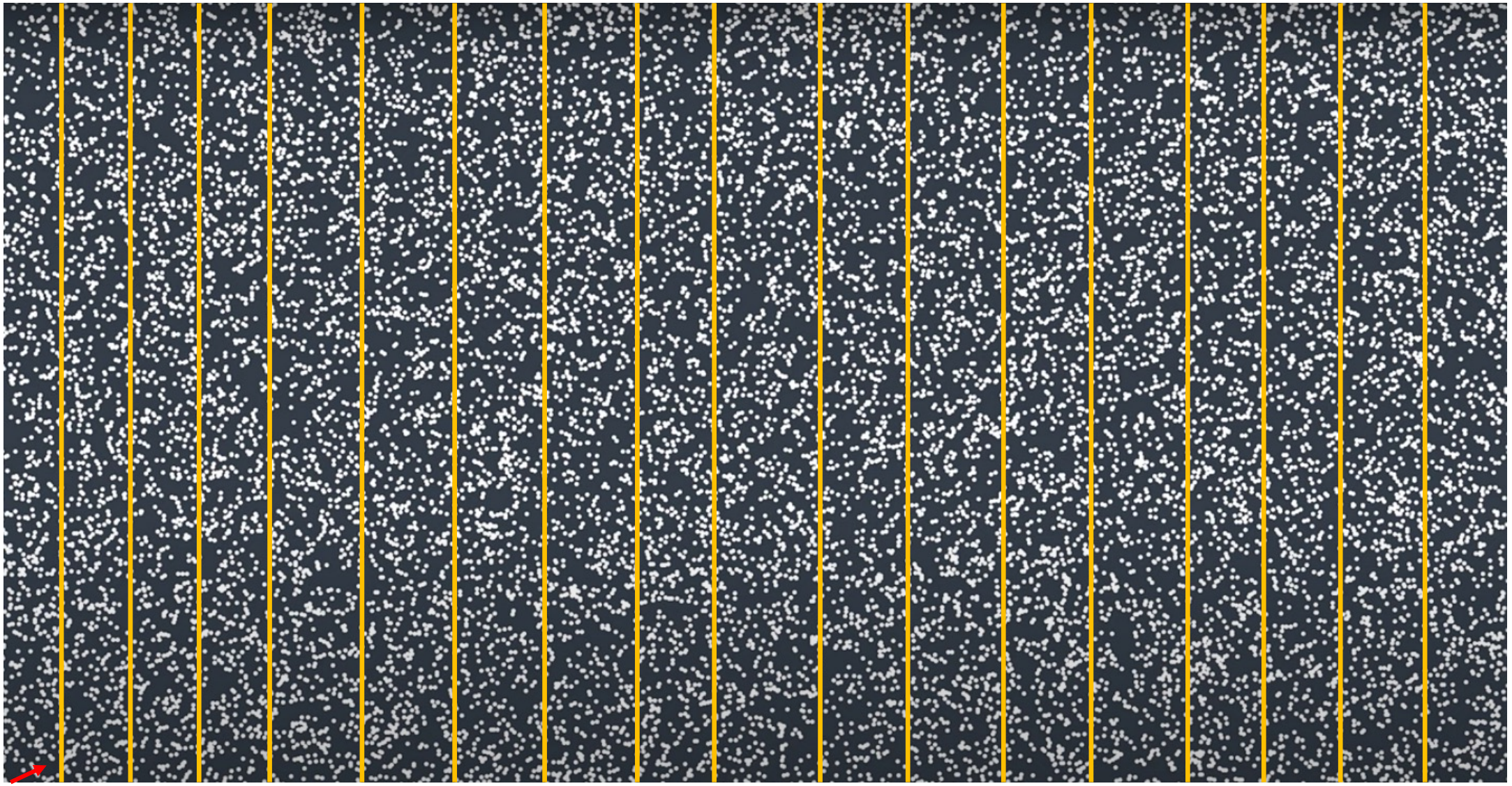
Closest-Pair Problem by Divide-and-Conquer



Closest-Pair Problem by Divide-and-Conquer

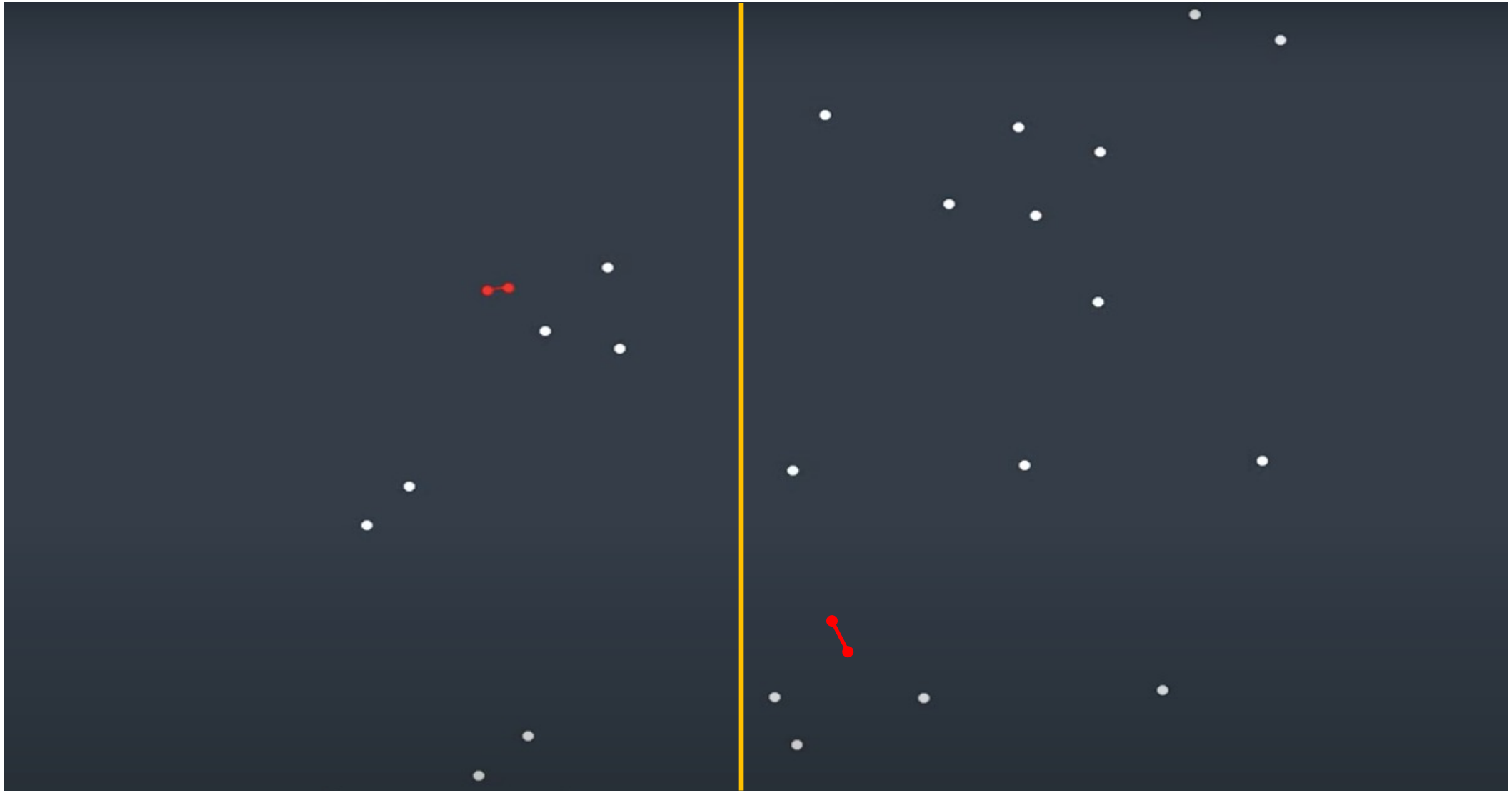


Closest-Pair Problem by Divide-and-Conquer

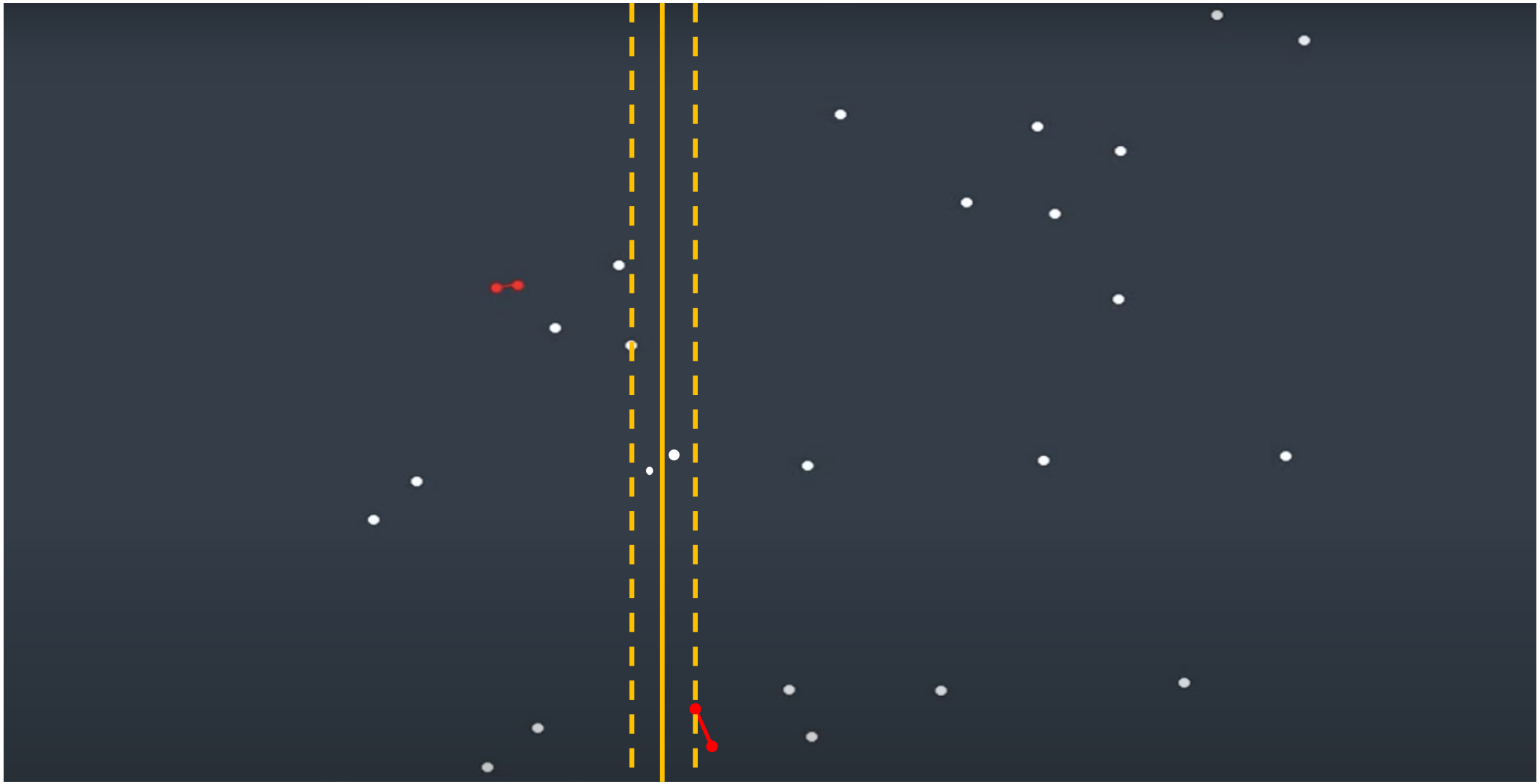


Closest-Pair Problem by Divide-and-Conquer

Closest-Pair Problem by Divide-and-Conquer

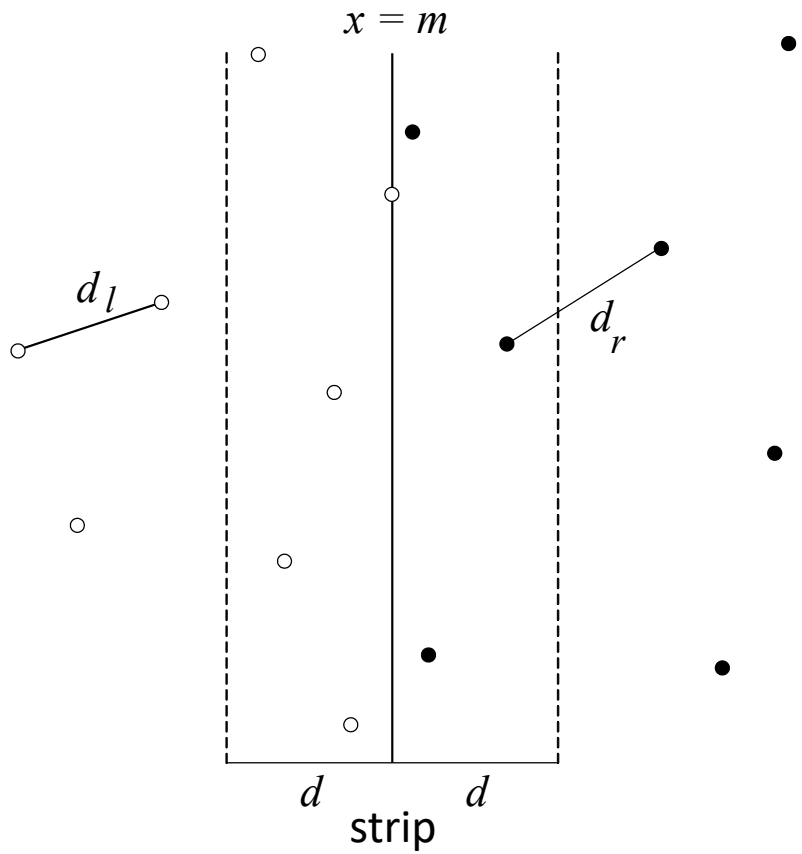


Closest-Pair Problem by Divide-and-Conquer



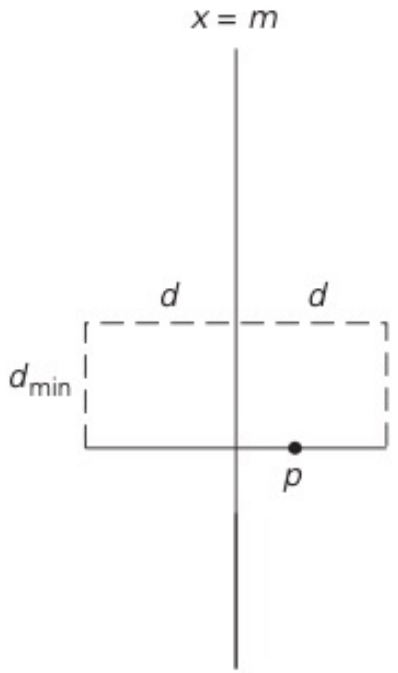
Closest-Pair Problem by Divide-and-Conquer

Step 1: Divide the points given into two subsets P_l and P_r by a vertical line $x = m$ so that half the points lie to the left or on the line and half the points lie to the right or on the line.



Closest-Pair Problem by Divide-and-Conquer

- Rectangle that may contain points closer than d_{min} to point p.



strip

Closest Pair by Divide-and-Conquer (cont.)

Step 2: Find recursively the closest pairs for the left and right subsets.

Step 3: Set $d = \min\{d_l, d_r\}$

- We can limit our attention to the points in the symmetric vertical strip S of width 2d as possible closest pair.
- The points are stored and processed in increasing order of their y coordinates.

Step 4: Scan the points in the vertical strip *S* from the lowest up.

- For every point $p(x,y)$ in the strip, inspect points in the strip that may be closer to p than d.

Efficiency of the Closest-Pair Algorithm

- The algorithm spends linear time both for dividing the problem into two problems half the size and combining the obtained solutions.
- Running time of the algorithm is described by

$$T(n) = 2T(n/2) + f(n), \text{ where } f(n) \in \Theta(n) \text{ (Finding median)}$$

- By the Master Theorem (with $a = 2$, $b = 2$, $d = 1$)

$$T(n) \in \Theta(n \log n).$$

Master Theorem
If $a < b^d$, $T(n) \in \Theta(n^d)$
If $a = b^d$, $T(n) \in \Theta(n^d \log n)$
If $a > b^d$, $T(n) \in \Theta(n^{\log_b a})$

Summary

- Multiplication of large integers
- Strassen's matrix multiplication
- Closest-Pair Problem by Divide and Conquer

Supportive Materials

- *Introduction to The Design and Analysis of Algorithms (3rd Edition) by Anany Levitin, Pearson.*
- *Introduction to Algorithms*, by T.Cormen, C. Leiserson, R.Rivest and C.Stein, MIT Press, 3rd Edition.